

Artificial Intelligence Homework Set 1

Your Name

March 2, 2026

Problem 1: INTRO 1

Question: What does the word "rationally" mean? Give 1 or 2 other concrete examples.

Answer: In the context of artificial intelligence, to "act rationally" means taking actions that are expected to maximize a specific performance measure or goal, given the available information and constraints.

Examples:

1. An autonomous driving system applying its brakes when a pedestrian suddenly steps into the road, maximizing the goal of safety and avoiding a collision.
2. A chess-playing AI program calculating and choosing a move that maximizes its probability of winning the game based on the current board state.

Problem 2: Intro 2

Question: Can you think of a task humans don't do well.

Answer: Humans do not perform well at processing and finding hidden patterns in massive datasets (Big Data) quickly. Another example is precisely comparing thousands of biometric features simultaneously, such as accurately checking and matching complex facial recognition features or fingerprints across a database of millions of records in mere seconds.

Problem 3: BFS 1

Question: Breadth-First Search on 8-puzzle. How many states the algorithm have searched to get the goal?

Answer: Based on the provided search tree diagram where the goal node is identified as node 27, the algorithm has searched 27 states to reach the goal. Theoretically, if a tree has a branching factor of b and we assume on depth level 5 the algorithm searched half of the states, the total number of searched states can be formulated as $N = \sum_{d=0}^4 b^d + \frac{b^5}{2}$.

Problem 4: DFS 1

Question: Draw the DFS search tree on this graph.

Answer: Based on the entry and exit times provided in the reference diagram (e.g.,

u: 1/12, v: 2/11, t: 3/10, s: 4/5, r: 6/9, w: 7/8), the sequence of Depth-First Search exploration forms the following tree edges:

- (u, v)
- (v, t)
- (t, s) [Backtrack from s to t]
- (t, r)
- (r, w)

The complete path of discovery is: $u \rightarrow v \rightarrow t \rightarrow s$, backtrack to t , then $t \rightarrow r \rightarrow w$, and finally backtrack all the way to u .

Problem 5: DFS 2

Question: Mark the entering and leave time for every nodes.

Answer: Following the DFS search tree provided (starting at root S):

- S: 1 / 16
- A: 2 / 15
- B: 3 / 14
- C: 4 / 5
- E: 6 / 13
- D: 7 / 8
- F: 9 / 12
- G: 10 / 11

Problem 6: DFS 3

Question: Show how DFS visit all nodes in the following graph. Assume alphabetical order.

Answer: Starting at node q , the DFS visits adjacency lists alphabetically. The sequence of discovered nodes is:

$$q \rightarrow s \rightarrow v \rightarrow w \rightarrow t \rightarrow x \rightarrow z \rightarrow y \rightarrow r \rightarrow u$$

Trace details: q goes to s , s to v , v to w , w to s (visited). Backtrack to q . q goes to t , t to x , x to z , z to x (visited). Backtrack to t . t goes to y , y to q (visited). Backtrack to q . q 's neighbor w is visited. Unvisited nodes are r and u . Start at r (alphabetically), r goes to u . All nodes visited.

Problem 7: Dijkstra 1

Question: Complete the Dijkstra algorithm on this graph.

Answer: Starting with source node a , the shortest distances to each node are updated step-by-step:

1. Init: $S = \{a\}$. $dist(a) = 0$, others ∞ .
2. Pick b (dist 4). $S = \{a, b\}$. Update h : $dist(h) = 4 + 1 = 5$. Update c : $dist(c) = 4 + 8 = 12$.
3. Pick h (dist 5). $S = \{a, b, h\}$. Update g : $dist(g) = 5 + 1 = 6$. Update i : $dist(i) = 5 + 7 = 12$.
4. Pick g (dist 6). $S = \{a, b, h, g\}$. Update f : $dist(f) = 6 + 2 = 8$.
5. Pick f (dist 8). $S = \{a, b, h, g, f\}$. Update e : $dist(e) = 8 + 10 = 18$. Update d : $dist(d) = 8 + 14 = 22$.
6. Pick c (dist 12). Update d : $dist(d) = \min(22, 12 + 7) = 19$.
7. Pick i (dist 12). No better paths.
8. Pick e (dist 18). No better paths.
9. Pick d (dist 19).

Final shortest distances: $a = 0, b = 4, h = 5, g = 6, f = 8, c = 12, i = 12, e = 18, d = 19$.

Problem 8: A* 1

Question: Run A* on the graph. A is start, T is goal. Compute $g(n)$ and $f(n)$.

Answer:

- **Iter 1:** Pop $A(g = 0, f = 30)$. Expand X . Open = $\{X(17, 44)\}$.
- **Iter 2:** Pop X . Expand B, C . $g(B) = 17 + 16 = 33, f(B) = 33 + 16 = 49$. $g(C) = 17 + 11 = 28, f(C) = 28 + 17 = 45$. Open = $\{C(45), B(49)\}$.
- **Iter 3:** Pop C . Expand L, W . $g(L) = 48, f(L) = 81$. $g(W) = 28 + 11 = 39, f(W) = 39 + 9 = 48$. Open = $\{W(48), B(49), L(81)\}$.
- **Iter 4:** Pop W . Expand K, T . $g(K) = 49, f(K) = 68$. $g(T) = 39 + 9 = 48, f(T) = 48 + 0 = 48$. Open = $\{T(48), B(49), K(68), L(81)\}$.
- **Iter 5:** Pop $T(48)$. Goal reached!

Optimal Path: $A \rightarrow X \rightarrow C \rightarrow W \rightarrow T$. Total Cost: 48.

Problem 9: A* 2

Question: On node B, $h = 16$ has been changed to $h = 6$. Can you get the same optimal path? Is there some problem with the algorithm?

Answer: Yes, you still get the exact same optimal path ($A \rightarrow X \rightarrow C \rightarrow W \rightarrow T$ with cost 48) because $h = 6$ is still an *admissible* heuristic ($6 \leq 17$, where 17 is the true minimal cost from B to T).

The Problem: The heuristic function is now *inconsistent* (or non-monotonic). For nodes X and B , consistency requires $h(X) \leq c(X, B) + h(B)$. However, $27 \not\leq 16 + 6$. Because it is inconsistent, the algorithm might extract nodes sub-optimally from the open list (e.g., extracting B before discovering a better path to W via C), leading to unnecessary expansions or forcing the algorithm to re-evaluate nodes that were already placed in the closed list.

Problem 10: A* 3

Question: (a) How many states? (b) Why shortest path goes along the edge of a polygon?

Answer:

- (a) The states in this pathfinding problem consist of the start state S , the goal state G , and all the vertices of the polygonal obstacles. Counting the vertices of the given shapes ($5 + 3 + 4 + 3 + 4 + 6 + 3 = 28$), plus S and G , there are exactly **30 states**.
- (b) In Euclidean space, the shortest distance between any two points is a straight line. When polygonal obstacles block the direct line of sight, the shortest valid path must bend around the obstacles. Due to the triangle inequality, any path that curves or bends at points other than the exact vertices of the obstacles will be strictly longer. Therefore, the path behaves like a taut rubber band, pulling tight directly against the edges/vertices of the polygons (forming a Visibility Graph).